

系统开发工具基础

第 1 周课后实验报告

姓名：彭俊皓 学号：25020007100

2026 年 8 月 31 日

仓库：<https://github.com/pjh456/sys-devtool/>

本报告覆盖第 1 周课堂四道实验（week1/q01-q04）的操作过程与结果，以及 missing-semester 《Shell 入门》课后练习第 1-12 题（脚本与输出均在 homework/course-shell/ 下可复现）；全部结果均在本机重新执行验证。

表 1: 第 1 周课后实验完成情况

部分	内容	关键工具	状态
一	课堂实验回顾（q01-q04）	Shell、awk、Git、LaTeX	🔵 完成
二	Shell 入门课后练习（第 1-12 题）	bash、find、chmod、管道	🟢 完成

一

课堂实验回顾（q01-q04）

q01

含空格文件名的批量整理

📖 题目要求

- 创建 input/docs、input/tmp 及含空格文件名 notes one.txt 等；
- 把 input 下全部 .txt 复制到 work/25020007100/，保留相对目录结构；
- 目录权限 750、文件权限 640，生成 inventory.txt（相对路径 + 字节数）。

>_ 关键命令

```
1 mkdir -p input/docs input/tmp
2 printf 'alpha\nbeta\n' > "input/docs/notes one.txt"
3 touch input/tmp/empty.txt
4 find . -type f -name '*.txt' ! -name '.secret.txt' \
5   -exec cp --parents {} ../work/25020007100/ \;      # 复制，保留相对结构
6 find work/25020007100 -type d -exec chmod 750 {} + # 目录 750
7 find work/25020007100 -type f ! -name inventory.txt \
8   -exec chmod 640 {} +                               # 文件 640
```

```

9 find . -type f -name '*.txt' ! -name 'inventory.txt' \
10 -printf '%s %p\n' | sed 's|\.|work/25020007100/|' > inventory.txt

```

✓ 结果与验证

```

$ cat work/25020007100/inventory.txt
11 work/25020007100/docs/notes one.txt
0 work/25020007100/tmp/empty.txt

```

空格文件名完整保留；docs、tmp 为 drwxr-x--- (750)，notes one.txt、empty.txt 为 -rw-r----- (640)。

💡 要点与注意事项

含空格路径全程加引号；复制用 `cp --parents`、统计用 `find -printf`，二者都不按空格分词，避免文件名被截断。

q02 随机访问日志统计

📖 题目要求

编写 `analyze.sh`：输入不存在的文件时写 `stderr` 并返回非零退出码；输出 5xx 数量最多的前 2 个 path（次数降序、path 字典序）；输出平均 `latency_ms`（两位小数，表头不计入）；不得使用 Python。

🔗 关键管道

```

1 # 前 2 个 5xx path: awk 按表头定位列 -> sort|uniq -c 计数 -> 次数降序+名称升序 -> head -2
2 awk -F " , " 'NR==1{for(i=1;i<=NF;i++)h[$i]=i}
3         NR>1 && $h["status"] ~ /^5[0-9]{2}$/{print $h["path"]}' "$path" |
4     sort | uniq -c | sort -k1,1rn -k2,2 | sed 's/^ *//' | head -2
5 # 平均延迟: 动态定位列, 求和除以数据行数 NR-1
6 awk -F " , " 'NR==1{for(i=1;i<=NF;i++)h[$i]=i}
7         NR>1 {sum+=$h["latency_ms"]}}
8         END {printf "%.2f\n", sum/(NR-1)}' "$path"

```

✓ 结果与验证

```

$ sh analyze.sh access.csv
3 /api/orders
2 /api/users
193.75

```

```

$ sh analyze.sh nope.csv
nope.csv is not found      # 写入 stderr, 退出码 1

```

平均延迟核算: $(120 + 310 + 180 + 90 + 260 + 20 + 420 + 150)/8 = 1550/8 = 193.75$ 。

💡 要点与注意事项

`uniq -c` 前必须先 `sort`; 次序要求用 `sort -k1,1rn -k2,2` (次数数值降序 + path 字典序) 实现; `awk` 按表头定位列而非硬编码列号, 脚本更健壮。

q03 制造、解决并解释一次合并冲突

📖 题目要求

在 q03 仓库从零初始化: `main` 上创建 `config.txt` (`mode=normal`); 从初始 `main` 建 `feature-a` (`mode=safe`)、`feature-b` (`mode=fast`); 回 `main` 先合 `a` 再合 `b`, 解决冲突保留 `mode=safe` 并加一行 `note=reviewed`。

git 操作过程

```

1 git init
2 printf 'mode=normal\n' > config.txt
3 git add config.txt && git commit -m "main branch first commit"
4 git checkout -b feature-a
5 printf 'mode=safe\n' > config.txt
6 git commit -am "feature-a branch config changed"
7 git checkout main
8 git checkout -b feature-b
9 printf 'mode=fast\n' > config.txt
10 git commit -am "feature-b branch config changed"
11 git checkout main
12 git merge feature-a          # 顺利合并 -> mode=safe
13 git merge feature-b         # 冲突: 两边都改了同一行
14 printf 'mode=safe\nnote=reviewed\n' > config.txt
15 git add config.txt && git commit

```

✅ 结果与验证

最终 `config.txt` 为 `mode=safe + note=reviewed`; `git log --all --graph --oneline` 呈菱形历史, 合并提交 `4538fe4` 为 `main HEAD`, 工作区干净。

```

*    4538fe4 (HEAD -> main) Merge feature-b and ignore its change
|\
| * 1c5d24b (feature-b) feature-b branch config changed
* | 45a9ccc (feature-a) feature-a branch config changed
|/
* 8e50da7 main branch first commit

```

💡 要点与注意事项

两分支在基线之后修改同一行必然冲突；解决的本质是手工判定保留哪一侧（此处保留 safe 并补充 note 行），再完成合并提交。

q04 修复并构建一页技术说明

📖 题目要求

补全 q04/report.tex: 加入带编号与 label 的公式并用 \eqref 引用；加入 2×2 表格 (caption + label) 并在正文引用；构建 report.pdf，交叉引用不得出现“??”。

⚡ 构建与验证

```
1 pdflatex -interaction=nonstopmode report.tex # 本机无 latexmk, 两遍编译使
2 pdflatex -interaction=nonstopmode report.tex # \label/\ref 编号收敛
3 pdftotext report.pdf - | grep -c '??' # 输出 0, 无未定义引用
```

✅ 结果与验证

pdftotext 验证：正文显示“公式 (1) 是一个重要的等式”“表 1 展示了实验数据”，公式与表格编号均正确解析，无“??”。

💡 要点与注意事项

\eqref 自动带括号、\ref 仅出编号；交叉引用依赖 .aux，需编译两遍收敛。

二 课后练习：missing-semester 《Shell 入门》第 1–12 题

第 1 题 确认当前 Shell 环境

📖 题目要求

课程要求使用类 Unix Shell (Bash/Zsh 等)。运行 echo \$SHELL，输出类似 /bin/bash 或 /usr/bin/zsh 即满足要求。

```
1 echo $SHELL
```

✓ 结果与验证

```
$ sh 1.sh
/usr/bin/bash
```

当前默认 Shell 为 bash，满足课程对 Shell 的要求，后续练习均在此环境完成。

第 2 题 `ls -l` 长格式与权限位

📖 题目要求

`ls` 的 `-l` 选项作用是什么？运行 `ls -l /` 并观察输出，每一行最前面的 10 个字符分别代表什么？（提示：man `ls`）

>_ 操作 (2/`ls.sh`、2/`manls.sh`)

```
1 ls -l /
2 man ls
```

✓ 结果与验证

```
$ ls -l /
总计 52
lrwxrwxrwx   1 root root      7 2025年10月13日 bin -> usr/bin
drwx-----  5 root root  4096 1970年 1月 1日 boot
drwxr-xr-x  20 root root  4480 8月31日 13:29 dev
drwxr-xr-x 110 root root  4096 8月31日 13:30 etc
..... (共 50 余行)
```

`-l` 为长格式：按行列出每个文件的类型与权限、链接数、属主、属组、字节数、修改时间与名称。每行最前 10 个字符的含义：

- 第 1 个字符为文件类型：d 目录、- 普通文件、l 符号链接；
- 后 9 个字符按 3 个一组，分别是属主、属组、其他人的读 r/写 w/执行 x 权限。

例如 `drwx-----`：目录，仅属主可读写进入；`lrwxrwxrwx`：链接，任何人可读写。

第 3 题 `glob`（文件名展开）

📖 题目要求

`find` 命令中的 `"*.zip"` 是一个 `glob`。什么是 `glob`？新建一个测试目录并创建一些文件，试试 `ls *.txt`、`ls file?.txt`、`ls {a,b,c}.txt` 等模式。

➤ 操作 (3/ 目录)

新建目录 `homework/course-shell/3`, 其中放入 `a.txt`、`b.txt`、`c.txt`、`data.csv`、`json/toml`、`dir1.txt`、`dir2.txt`、`file1--3.txt`, 然后分别运行:

```
1 # lsstar.sh
2 ls *.txt
3 # lsfile.sh
4 ls file?.txt
5 # lsscope.txt (命令行直接运行)
6 ls {a,b,c}.txt
```

✓ 结果与验证

```
$ ls *.txt
a.txt  b.txt  c.txt  dir1.txt  dir2.txt
file1.txt  file2.txt  file3.txt  lsscope.txt

$ ls file?.txt
file1.txt  file2.txt  file3.txt

$ ls {a,b,c}.txt
a.txt  b.txt  c.txt
```

glob (文件名展开) 是 shell 在调用命令之前自行完成的模式匹配: shell 先用该模式匹配当前目录下的文件名, 把匹配结果作为一个参数列表展开后再传给命令。三种模式的区别:

- `*` 匹配任意长度 (含空) 的字符, 故 `*.txt` 匹配所有以 `.txt` 结尾的文件;
- `?` 恰好匹配一个字符, 故 `file?.txt` 只匹配 `file1--3.txt` 这类 “file+ 一位+.txt” 的名字;
- `{a,b,c}.txt` 是大括号展开, 等价于 `ls a.txt b.txt c.txt`, 与 `*/*` 的 “匹配” 不同, 它是直接生成多个候选字符串。

若模式无匹配项, `bash` 默认原样传递该模式 (可设 `nullglob` 使其展开为空)。

第 4 题 三种引号

📖 题目要求

' 单引号'、" 双引号" 和 `$'ANSI 引号'` 有什么区别? 写一条命令, 输出一个同时包含字面量 `$`、`!` 和换行符的字符串。

🔗 解答 (4.sh)

```
1 echo "\$\"'!'$'\n'
```

✓ 结果与验证

```
$ sh 4.sh | od -c
00000000  $   !   \n   \n
00000004
```

输出即 `$!` 加换行（末尾 `\n` 由 `echo` 自带，`od -c` 验证字节内容无误）。该命令把三种引号各用了一段：

- `"$"`：双引号内允许变量与命令展开，`$` 加反斜杠即输出字面量 `$`；
- `'!'`：单引号内一切皆字面量，避免交互式 `bash` 把 `!` 当作历史扩展；
- `$'\n'`：ANSI-C 引号，解释 `\n` 转义得到真正的换行。

三者对比：单引号完全不展开；双引号展开 `$var`、`$(cmd)` 与 `\$` `\\\`` `\"` 等少数反斜杠序列；`$'...'` 解释 C 风格转义（`\n`、`\t`、`\xNN` 等）。

第 5 题 标准流重定向

📖 题目要求

Shell 有三条标准流：`stdin(0)`、`stdout(1)`、`stderr(2)`。运行 `ls /nonexistent /tmp`，把 `stdout` 和 `stderr` 分别重定向到两个文件；再说明如何把两者都重定向到同一个文件。

➤ 操作（5/rd2.sh、5/rd1.sh）

```
1 # 分流：stdout 与 stderr 各进一个文件
2 ls /nonexistent /tmp 1> stdout.log 2> stderr.log
3 # 合并：先让 stdout 进 all.log，再让 stderr 跟随 stdout (2>&1 顺序不可颠倒)
4 ls /nonexistent /tmp > all.log 2>&1
```

✓ 结果与验证

三个日志文件内容（/tmp 列表较长，节选首行）：

```
$ cat stderr.log
"/nonexistent": No such file or directory (os error 2)

$ head -3 stdout.log
37f59b10-0c81-4765-a66a-fec19b4fb3c6.png
148e3933-7014-4822-983a-6b0af7a4afb4.png
fdf03fa1-4baa-4b2a-9806-8349bc305799.png

$ head -2 all.log
"/nonexistent": No such file or directory (os error 2)
37f59b10-0c81-4765-a66a-fec19b4fb3c6.png
```

分流写法中 1> 与 2> 各自独立指向文件，两路互不干扰；合并写法 > all.log 2>&1 中 2>&1 必须写在 > 之后：先使 fd1 指向 all.log，再让 fd2 “跟随”当前的 fd1，二者才指向同一文件；若写成 2>&1 > all.log，stderr 会先指向终端，stdout 才改道，错误信息就丢不进文件了。

第 6 题 一行命令：仅在不存在时创建

题目要求

\$? 保存上一条命令的退出状态（0 为成功）；&& 仅在前一条成功时执行后一条，|| 仅在前一条失败时执行后一条。写一行命令：仅当 /tmp/mydir 不存在时才创建它。

</> 解答 (6.sh)

```
1 ls /tmp/mydir 2> /dev/null || mkdir -p /tmp/mydir
```

结果与验证

```
$ sh 6.sh # 目录不存在: ls 失败(退出码 2) -> || 触发 mkdir
$ ls -ld /tmp/mydir
drwxr-xr-x 2 pjh123 pjh123 40 8月31日 14:40 /tmp/mydir
$ sh 6.sh # 目录已存在: ls 成功, || 短路, 不再创建
$ echo $?
0
```

ls 对不存在的目录返回非零退出码，于是 || 右侧的 mkdir -p 被执行；目录已存在时 ls 成功、|| 短路跳过，命令具备幂等性。stderr 丢弃到 /dev/null 避免把错误信息打印到终端。

第 7 题 为什么 cd 是内建命令

题目要求

为什么 cd 必须是 Shell 内建命令，而不能是独立程序？（提示：想想子进程能影响和不能影响父进程的哪些状态。）

结果与验证

答：(7.txt)

- cd 的作用是修改当前 shell 进程自身的工作目录；
- 若 cd 是独立程序，shell 必须先 fork 出子进程再 exec 它，子进程切换的只是它自己的工作目录；

- 子进程退出后其状态（包括 cwd 变化）不会传回父进程，对父进程毫无影响；
- 因此 cd 必须由 shell 进程内部直接实现，即内建命令，才能就地修改 shell 自己的 cwd，使后续命令在新的目录下解析相对路径。

第 8 题 检查文件是否存在

📖 题目要求

写一个脚本，接收文件名参数 (\$1)，用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在，并根据结果输出不同提示。

🔗 解答 (8.sh)

```
1 #!/bin/sh
2 if [ $# -lt 1 ]; then
3     echo "File Path is required!" >&2
4     exit 1
5 fi
6 FP=$1
7 if [ -f $FP ]; then
8     echo "File exists"
9     exit 0
10 else
11     echo "File doesn't exist"
12     exit 1
13 fi
```

✅ 结果与验证

```
$ sh 8.sh 7.txt
File exists
$ echo $?
0
$ sh 8.sh nope.txt
File doesn't exist
$ echo $?
1
```

无参数时写 stderr 并以 1 退出；存在/不存在分别以 0/1 退出码区分，便于脚本间串联判断。

第 9 题 可执行权限

📄 题目要求

把第 8 题脚本保存为文件，先运行 `./8.sh somefile`，会发生什么？然后执行 `chmod +x 8.sh` 再试一次。为什么这一步是必须的？（提示：比较 `chmod` 前后的 `ls -l 8.sh`。）

✅ 结果与验证

```
$ ./8.sh 8.sh
bash: ./8.sh: 权限不够
$ echo $?
126
$ ls -l 8.sh
-rw-r--r-- 1 pjh123 pjh123 176 8月31日 8.sh
$ chmod +x 8.sh
$ ls -l 8.sh
-rwxr-xr-x 1 pjh123 pjh123 176 8月31日 8.sh
$ ./8.sh 8.sh
File exists
```

答：Linux 内核只允许执行带可执行位的文件：`chmod` 前权限为 `-rw-r--r--`，没有 `x` 位，`execve` 被内核以 `EACCES` 拒绝，shell 报“权限不够”（退出码 126）；加上 `+x` (`-rwxr-xr-x`) 后内核放行，脚本才能作为命令直接运行。

第 10 题 set -x 调试输出

📄 题目要求

在脚本的 `set` 选项里加入 `-x` 会发生什么？写个简单脚本试试并观察输出。

🔗 解答 (10.sh)

```
1 #!/bin/sh
2 set -x
3 name='Noa'
4 echo "Hello, $name"
5 count=3
6 if [ $count -gt 1 ]; then
7     echo "count $(count) is greater than 1"
8 else
9     echo "count is not greater than 1"
10 fi
11 set +x
```

结果与验证

```
$ sh 10.sh
+ name=Noa
+ echo 'Hello, Noa'
Hello, Noa
+ count=3
+ '[' 3 -gt 1 ']'
++ count
usage: count <expected line count>
+ echo 'count is greater than 1'
count is greater than 1
+ set +x
```

`set -x` 开启 `xtrace`: shell 在执行每条命令之前, 把该命令连同变量/参数展开后的实际值以 + 前缀打印到 `stderr` (子命令用 `++`); `set +x` 关闭。从输出可以看到 `$(count)` 实际展开为空、`[` 的真实形式 `'[ ' 3 -gt 1 ' ]'` 等“幕后”细节, 这正是调试脚本中条件与展开是否如预期的手段。

第 11 题 带日期的备份脚本

题目要求

写一条命令/脚本, 把文件复制为带当天日期的备份文件名 (例如 `notes.txt` → `notes-2026-08-31.txt`)。 (提示: `$(date +%Y-%m-%d)`)

</> 解答 (11.sh)

```
1 #!/bin/sh
2 if [ $# -lt 1 ]; then
3     echo "File path is required!"
4     exit 1
5 fi
6 FP=$1
7 FPP=$(dirname "$FP")           # 所在目录
8 FNEXT=$(basename $FP)         # 文件名
9 FN="${FNEXT%.*}"               # 去掉后缀 -> 主名
10 EXT="${FNEXT##*.}"            # 最长前缀匹配 -> 扩展名
11 NFN="$FN-$(date +%Y-%m-%d).$EXT"
12 ABSNFP="$FPP/$NFN"
13 cp $FP $ABSNFP
14 echo "Copy $FP to $ABSNFP"
```

✓ 结果与验证

```
$ printf 'hello\n' > notes.txt
$ sh 11.sh notes.txt
Copy notes.txt to ./notes-2026-08-31.txt
$ cat notes-2026-08-31.txt
hello
```

参数展开 `$FNEXT%.*`（去掉最短的后缀 `.*`）与 `$FNEXT##*.`（剥掉最长的 `.*` 前缀）配合，可无损拆分“主名 + 扩展名”，再由 `date +%Y-%m-%d` 插入当天日期生成备份名。

第 12 题 参数化 flaky test 复现脚本

📖 题目要求

修改讲义中“复现偶尔才会失败的测试”脚本：使它能够从命令行参数接收测试命令，而不是在脚本中写死 `cargo test my_test`。（提示：`$1` 或 `$@`）

🔗 解答 (12.sh)

```
1  #!/bin/bash
2  set -euo pipefail
3
4  if [ $# -eq 0 ]; then
5      echo "Test command is required!"
6      exit 1
7  fi
8  CMD="$*" # 修改点：测试命令改为从参数接收
9
10 # Start CPU stress in background
11 stress --cpu 8 &
12 STRESS_PID=$!
13
14 # Setup log file
15 LOGFILE="test_runs_$(date +%s).log"
16 echo "Logging to $LOGFILE"
17
18 # Run tests until one fails
19 RUN=1
20 while eval "$CMD" > "$LOGFILE" 2>&1; do # 修改点：eval 执行外部命令
21     echo "Run $RUN passed"
22     ((RUN++))
23 done
24
25 # Cleanup and report
26 kill $STRESS_PID
27 echo "Test failed on run $RUN"
```

```

28 echo "Last 20 lines of output:"
29 tail -n 20 "$LOGFILE"
30 echo "Full log: $LOGFILE"

```

相对讲义原脚本的修改：1. 开头增加参数校验（`$# -eq 0` 时报错退出）；2. 用 `CMD="$@"` 收集全部参数、`eval "$CMD"` 执行，替代写死的 `cargo test my_test`；3. `set -euo pipefail` 严格模式防坑。

✓ 结果与验证

选取一条“第 1 次成功、第 2 次失败”的命令验证循环逻辑：

```

$ bash 12.sh '[ ! -f /tmp/w1mer_flag ] && touch /tmp/w1mer_flag'
Logging to test_runs_1788158444.log
Run 1 passed
Test failed on run 2
Last 20 lines of output:
Full log: test_runs_1788158444.log

```

第 1 轮 `[ ! -f ... ]` 为真并落盘标记，测试通过；第 2 轮标记已存在，命令退出码 1，`while` 条件失败退出循环，脚本报告失败轮次并打印日志尾部，与讲义脚本行为一致，且测试命令完全由调用方指定。

三 总结与收获

• 课堂实验（q01–q04）

- 空格文件名要靠引号与 `cp --parents`、`find -printf` 等不分词方案处理。
- `sort | uniq -c` 配合多键排序即可完成“计数 + 多准则排序”。
- Git 冲突解决的本质是手工裁决保留哪一侧，再补上补充内容。
- LaTeX 交叉引用依赖两遍编译才能收敛编号。

• 课后练习（第 1–12 题）

- `glob` 是 shell 在调用命令前完成的文件名展开，`*`、`?` 与大括号各有语义。
- 单引号、双引号、`$'...'` 三种引号控制不同的展开程度。
- 三条标准流可独立分流，`> f 2>&1` 的书写顺序决定合并是否正确。
- `$?` 与 `&&`、`||` 可组合成一行式控制流。
- `cd` 必须内建、脚本必须 `chmod +x`，都源于“状态与权限只作用于自身进程”的进程模型。
- `set -x` 打印展开后的真实命令，是调试脚本展开与条件走向的利器。
- `$var%.*` 与 `$var##*`。参数展开可以不借助外部命令拆分文件名。